South Africa Household Survey Analysis (SDHSA)

Milestone 2 — Data Preparation Report

Table of Contents

Table of Contents 2
1. Introduction 4
2. Task 1 — Confirm Data Selection (Person 1) 4
2.1 Reconfirmed Dataset Selection 4
2.2 Target Analytical Grain 4
2.3 Attribute Weighting Approach 4
2.4 Relevant Subset Selection 5
3. Task 2 — Clean and Recode (Person 1) 6
3.1 Province Casing (applies to all five datasets) 6
3.2 D01 — Population Profile 6
3.2.1 Missing HouseholdID 6
3.2.2 Invalid Ages 6
3.2.3 Duplicate Person-Keys 7
3.3 D04 — Dwelling Conditions 7
3.3.1 Exact Duplicate Rows 7
3.3.2 RentRaw Sentinel 7
3.3.3 WallMaterialCode — Missingness and Sentinel 8
3.3.4 HouseholdSize == 0 8
3.3.5 TotalRooms Sentinel 8
3.4 D05 — Basic Services 9
3.4.1 Orphan Household IDs 9
3.4.2 Sentinel Codes in Water/Toilet Fields 9
3.4.3 DrinkingWaterSourceCode Missingness 9
3.5 D07 — Household Wellbeing (Outcome Dataset) 10
3.5.1 Exact Duplicate Rows 10
3.5.2 Food-Security Item Sentinels 10
3.5.3 HouseholdSize == 0 10
3.5.4 Income Sentinel Family and Extreme Outliers 10
3.5.5 RanOutOfFoodCode Recoding 11
3.6 D09 — Household Resources 11
3.6.1 Exact Duplicate Rows 11
3.6.2 MainIncomeSourceCode 12
3.6.3 TotalVehicles Mixed Encoding 12

3.6.4 TotalMonthlyHouseholdIncomeRaw 12
3.7 Cross-File Consistency Checks (Diagnostic Only) 13
3.8 Post-Cleaning Verification Summary 13
4. Task 3 — Integrate and Construct (Person 2) 14
4.1 Inputs 14
4.2 Target Grain and Integration Strategy 14
4.3 Aggregate D01 to Household Grain 14
4.4 Confirm Join Cardinality Before Joining 14
4.5 Sequential Joins with Row-Count Verification 15
4.6 Column-Level Selection: Dropping Redundant Raw Income Copies 15
4.7 Derived / Engineered Features 16
4.7.1 Services_access_index 16
4.7.2 Income_per_capita 16
4.7.3 FoodInsecure_flag 16
4.8 Final Data Dictionary 17
4.9 Export ..... **Error! Bookmark not defined.**
5. Task 4 — Prepare Data for Modelling (Person 3) 20
5.1 Define the Modelling Target 20
5.2 Assess Class Imbalance 20
5.3 Remove Identifiers and Low-Value Columns 20
5.4 Remove Leakage Variables 20
5.5 Transformations and Encoding 20
5.6 Scaling 20
5.7 Train/Test Split 21
5.8 Leakage Prevention Check 21
6. Summary and Outstanding Items 22

1. Introduction

This report documents the data-preparation work carried out for Milestone 2 of the SDHSA project, covering Task 1 (confirm data selection), Task 2 (clean and recode) and Task 3 (integrate and construct). Task 1 and Task 2 were carried out by Person 1 on the five source datasets selected in Milestone 1: D01_Population_Profile, D04_Dwelling_Conditions, D05_Basic_Services, D07_Household_Wellbeing and D09_Household_Resources. Task 3 was carried out by Person 2, building directly on Person 1's cleaned exports to produce a single, integrated, household-level analytical dataset with engineered features and a data dictionary.

For every major preprocessing decision below, four things are documented consistently: the Problem identified in the raw data, the Treatment (method or transformation) applied in R, the Effect the transformation had on the dataset, and the Verification (code output, count or summary) that confirms the fix worked as intended. All row counts, NA counts and summary statistics quoted below are taken directly from the executed R output in each author's rendered R Markdown report.

2. Task 1 — Confirm Data Selection

2.1 Reconfirmed Dataset Selection

The Milestone 1 selection of D01_Population_Profile, D04_Dwelling_Conditions, D05_Basic_Services, D07_Household_Wellbeing and D09_Household_Resources is reconfirmed for Milestone 2, chosen to explain household food security / well-being via demographic composition, housing and service access, and household resources. Milestone 1's full 11-dataset inspection did not surface a reason to add or drop a dataset from this set. It did confirm that D11 is structurally incompatible with all others (zero shared HouseholdIDs) and would have been excluded even if it had been considered.

2.2 Target Analytical Grain

The final analytical dataset will be at household level (one row per HouseholdID). D04, D05, D07 and D09 are already at this grain. D01 is person-level and must be aggregated to household level (e.g. household size, dependants, population-group composition) before it can be joined — this aggregation is a Task 3 integration step, carried out by Person 2. Person 1's responsibility at this stage is to clean D01 at its native person-level grain and hand off a tidy person-level file plus a note on the aggregation needed.

2.3 Attribute Weighting Approach

Rather than assign arbitrary numeric weights before any model exists, attributes are tiered qualitatively to guide which variables Task 4 (Person 3) should prioritise as predictors versus context.

Tier	Meaning	Examples
High	Directly measures resources/access hypothesised to drive the outcome	MainIncomeSourceCode, SocialGrantRecipients (D09); DrinkingWaterSourceCode, MainToiletCode (D05); the food-security item codes (D07)
Medium	Demographic/context variables that condition or moderate the outcome	Age, PopulationGroupCode (D01); MainDwellingCode, TenureCode (D04)

Tier	Meaning	Examples
Low	Administrative/design fields — needed for correct estimation but not substantive predictors	SurveyYear, GeoTypeCode, *Weight (survey design weights)

High-tier variables are the primary candidates for Milestone 3 modelling/segmentation; Medium-tier variables are kept as demographic-segmentation/control variables; Low-tier variables are retained for correct survey-weighted estimation but are never used as predictors.

2.4 Relevant Subset Selection

Column-level selection is applied per dataset in Task 2, dropping columns that are pure skip-logic artefacts with no analytical content once decoded. Row-level selection (removing orphan/duplicate records) is also handled in Task 2, with before/after row counts as verification in each case (see Section 3).

3. Task 2 — Clean and Recode

All five files were read with HouseholdID forced to character type to avoid leading-zero loss. A province lookup table (ProvinceCode 1–9 mapped to the nine official province names) was built once and applied to all five datasets before the per-dataset cleaning steps below.

3.1 Province Casing (applies to all five datasets)

Problem: Province (text) is inconsistently cased, with up to 24 distinct values observed where only 9 are expected. ProvinceCode (numeric, 1–9) is confirmed clean and consistent across every file, so the text field cannot be trusted as-is.

Treatment: Derive a single canonical ProvinceClean label from ProvinceCode via a lookup table (province_lookup); rename the original Province column to Province_source and retain it for traceability rather than deleting it. Verified before applying the join that every ProvinceCode in every file falls in 1–9 with no missing codes, and that the modal Province text for each code matches the mapping exactly, so the join cannot silently introduce an NA province.

```
fix_province <- function(df) {
  df %>%
    rename(Province_source = Province) %>%
    left_join(province_lookup, by = "ProvinceCode")
}
```

Effect: All five datasets gain a clean ProvinceClean column resolving to exactly 9 categories instead of up to 24.

Verification: n_distinct(ProvinceClean) returns 9 for D01, D04, D05, D07 and D09. table(d01$ProvinceClean) confirms sensible household counts per province (Eastern Cape 9,340; Free State 3,924; Gauteng 16,716; KwaZulu-Natal 12,611; Limpopo 8,316; Mpumalanga 5,865; North West 4,135; Northern Cape 2,998; Western Cape 6,535).

3.2 D01 — Population Profile

3.2.1 Missing HouseholdID

Problem: 25 rows have no HouseholdID at all. These rows cannot be linked to any household and, left in place, distort every downstream key check (they directly cause the apparent duplicate-key issue addressed in §3.2.3).

Treatment: Exclude these rows from the working file and preserve them separately for audit in D01_excluded_missing_householdid.csv — dropped silently is not acceptable, dropped-and-logged is.

```
missing_id <- is.na(d01$HouseholdID) | d01$HouseholdID == ""
d01_excluded <- d01 %>% filter(missing_id)
write_csv(d01_excluded, "D01_excluded_missing_householdid.csv")
d01 <- d01 %>% filter(!missing_id)
```

Effect: D01's working row count drops from its raw import size to 70,415 person-level rows once the 25 ID-less rows are removed.

Verification: sum(is.na(HouseholdID) | HouseholdID == "") = 25 before the filter; nrow(d01) = 70,415 immediately after — matching the row count Person 2 later reports on import into Task 3.

3.2.2 Invalid Ages

Problem: Milestone 1 found 9 negative ages (as low as −7) and 9 ages above 120 (as high as 175) — 18 biologically impossible values in total.

Treatment: Recode biologically impossible ages (Age < 0 or Age > 110) to NA in a new Age_clean column, rather than guessing a true value or deleting the whole row — the rest of the row's data may still be usable.

```
before_invalid <- sum(d01$Age < 0 | d01$Age > 110, na.rm = TRUE)
d01 <- d01 %>% mutate(Age_clean = if_else(Age < 0 | Age > 110, NA_real_, Age))
```

Effect: Age_clean summary: Min 0, 1st Qu. 13, Median 28, Mean 30.49, 3rd Qu. 45, Max 110, with 108 NAs (the 18 recoded invalid values plus pre-existing genuine missing ages).

Verification: "Invalid ages before: 18 | after recode: 0" — the re-check sum(Age_clean < 0 | Age_clean > 110, na.rm = TRUE) returns 0, confirming no impossible values remain.

3.2.3 Duplicate Person-Keys

Problem: A naive count(HouseholdID, PersonNumber) on the file (while it still contained the 25 missing-ID rows) reported several 'duplicate' keys. Investigation showed every one of those groups shared a missing HouseholdID rather than a real one — dplyr::count() buckets all NAs together, so unrelated households that happen to share a PersonNumber collapse into one apparent group the moment their ID is blank. This was confirmed to be an artefact of §3.2.1, not a genuine data-quality issue in its own right.

Treatment: Having already removed the 25 missing-ID rows, the duplicate-key check is re-run on the remaining, ID-complete data, and a flag_duplicate_key column is kept regardless of the result so Task 3 has a documented check to rely on rather than re-deriving it.

```
dup_keys <- d01 %>% count(HouseholdID, PersonNumber) %>% filter(n > 1)
```

Effect: No rows are removed at this step; a diagnostic flag column (flag_duplicate_key) is added to D01.

Verification: nrow(dup_keys) = 0 and sum(flag_duplicate_key) = 0 once the missing-ID rows are gone, confirming HouseholdID + PersonNumber is a clean primary key for the remaining 70,415 rows.

3.3 D04 — Dwelling Conditions

3.3.1 Exact Duplicate Rows

Problem: 16 fully duplicate rows were found (confirmed in Milestone 1 by comparing full row content) — genuine copy duplicates, not legitimate repeated identifiers.

Treatment: Remove with distinct().

```
d04 <- d04 %>% distinct()
```

Effect: Row count falls from 20,956 to 20,940.

Verification: "D04 rows before: 20956 | after de-duplication: 20940 | removed: 16"; n_distinct(HouseholdID) == nrow(d04) returns TRUE, confirming exactly one row per household after de-duplication.

3.3.2 RentRaw Sentinel

Problem: The value 88 dominates RentRaw (75.2% of rows), meaning "not applicable" (household does not rent) rather than a real rent amount of R88.

Treatment: Recode 88 to NA in a new RentRaw_clean column; keep the original RentRaw for traceability.

```
d04 <- d04 %>% mutate(RentRaw_clean = if_else(RentRaw == 88, NA_real_, RentRaw))
```

Effect: 15,752 values move from the sentinel 88 to NA; RentRaw_clean summary drops to Min 1, 1st Qu. 2, Median 3, Mean 3.48, 3rd Qu. 5, Max 99, with 15,752 NAs.

Verification: sum(RentRaw == 88, na.rm = TRUE) = 15,752 and sum(is.na(RentRaw_clean)) = 15,752 — the counts match exactly, confirming every sentinel (and only the sentinel) was converted.

3.3.3 WallMaterialCode — Missingness and Sentinel

Problem: 55 genuine NAs (real missing values, not a sentinel issue), plus one further value of 99 that sits far outside the documented 1–12 category range and is itself a sentinel, not a wall material.

Treatment: Genuine NAs are left as NA — imputing a construction-material code with no domain basis would fabricate data; this is documented as low-volume missingness (0.26% of rows) that is acceptable to carry forward as-is. The 99 sentinel is recoded to NA separately in a new WallMaterialCode_clean column.

```
d04 <- d04 %>% mutate(WallMaterialCode_clean = if_else(WallMaterialCode == 99, NA_real_,
WallMaterialCode))
```

Effect: WallMaterialCode_clean has the single 99 sentinel converted to NA, in addition to the 55 pre-existing genuine NAs.

Verification: sum(is.na(WallMaterialCode)) = 55; sum(WallMaterialCode == 99, na.rm = TRUE) = 1 — both counts confirmed by direct code output before the recode.

3.3.4 HouseholdSize == 0

Problem: 12 households are recorded with zero members, which is not a valid household size by definition.

Treatment: Recode to NA in a new HouseholdSize_clean column rather than deleting the row — the rest of the dwelling record (materials, tenure, rooms) is still usable; only the size figure itself is untrustworthy.

```
d04 <- d04 %>% mutate(HouseholdSize_clean = if_else(HouseholdSize == 0, NA_real_,
HouseholdSize))
```

Effect: 12 values move from 0 to NA in HouseholdSize_clean; all other columns for those 12 rows are retained unchanged.

Verification: sum(HouseholdSize == 0, na.rm = TRUE) = 12, confirming the exact count nulled.

3.3.5 TotalRooms Sentinel

Problem: The raw maximum is 99 rooms (5 rows) — a clear outlier against the next-highest genuine value of 25, and consistent with the survey's 2-digit "not applicable/unspecified" sentinel pattern seen elsewhere in this file (e.g. RentRaw == 88).

Treatment: Recode 99 to NA in a new TotalRooms_clean column; values up to 25 are kept as plausible, since large households can genuinely have many rooms.

```
d04 <- d04 %>% mutate(TotalRooms_clean = if_else(TotalRooms == 99, NA_real_, TotalRooms))
```

Effect: TotalRooms_clean summary: Min 1, 1st Qu. 3, Median 5, Mean 4.96, 3rd Qu. 7, Max 25, with 5 NAs.

Verification: sum(TotalRooms == 99, na.rm = TRUE) = 5 before the recode, matching the 5 NAs reported in the post-recode summary().

3.4 D05 — Basic Services

3.4.1 Orphan Household IDs

Problem: 15 rows carry an UNMATCHED_xxxxx placeholder HouseholdID, confirmed in Milestone 1 to exist in no other selected dataset.

Treatment: Remove these rows — they cannot be joined to any other dataset and would only ever appear as unmatched records in Task 3, adding noise with zero analytical value for a household-level integrated dataset.

```
d05 <- d05 %>% filter(!str_starts(HouseholdID, "UNMATCHED"))
```

Effect: Row count falls from 20,940 to 20,925.

Verification: "D05 rows before: 20940 | after removing orphans: 20925 | removed: 15".

3.4.2 Sentinel Codes in Water/Toilet Fields

Problem: DistanceToWaterCode, PipedWaterCode, WaterInterruptionFrequencyCode, WaterAlternativeSourceCode and ToiletDistanceCode are each dominated by an 8 or 88 sentinel value ("not applicable").

Treatment: Recode 8 (or 88 for WaterAlternativeSourceCode) to NA in a new _clean column for each of the five fields.

```
d05 <- d05 %>% mutate(
  DistanceToWaterCode_clean = if_else(DistanceToWaterCode == 8, NA_real_,
DistanceToWaterCode),
  PipedWaterCode_clean = if_else(PipedWaterCode == 8, NA_real_, PipedWaterCode),
  WaterInterruptionFrequencyCode_clean = if_else(WaterInterruptionFrequencyCode == 8,
NA_real_, WaterInterruptionFrequencyCode),
  WaterAlternativeSourceCode_clean = if_else(WaterAlternativeSourceCode == 88, NA_real_,
WaterAlternativeSourceCode),
  ToiletDistanceCode_clean = if_else(ToiletDistanceCode == 8, NA_real_, ToiletDistanceCode)
)
```

Effect: Each field's _clean version has its sentinel population converted to NA: DistanceToWaterCode_clean and PipedWaterCode_clean each gain 16,733 NAs; WaterInterruptionFrequencyCode_clean gains 11,649; WaterAlternativeSourceCode_clean gains 15,343; ToiletDistanceCode_clean gains 20,241 (~96.7% of rows).

Verification: The NA counts produced by sapply(..., function(x) sum(is.na(x))) match the pre-recode sentinel counts identified in Milestone 1, e.g. ToiletDistanceCode_clean at ~96.7% NA is consistent with the original finding.

3.4.3 DrinkingWaterSourceCode Missingness

Problem: 60 genuine NAs found in DrinkingWaterSourceCode.

Treatment: Left as NA, using the same reasoning as D04's WallMaterialCode (§3.3.3) — genuine, low-volume missingness (0.29% of rows) with no safe basis for imputation.

Effect: No values changed; the column is documented as carrying a small, accepted amount of missingness.

Verification: 60 / 20,925 rows = 0.29%, calculated directly from sum(is.na(DrinkingWaterSourceCode)) against the cleaned D05 row count.

3.5 D07 — Household Wellbeing (Outcome Dataset)

3.5.1 Exact Duplicate Rows

Problem: 15 exact duplicate rows found.

Treatment: Remove with distinct().

```
d07 <- d07 %>% distinct()
```

Effect: Row count falls from 20,955 to 20,940.

Verification: "D07 rows before: 20955 | after de-duplication: 20940 | removed: 15".

3.5.2 Food-Security Item Sentinels

Problem: FewFoodsDays, SkippedMealDays, AteLessDays and RanOutDays are all dominated by the value 8, meaning "not applicable" (the household did not experience that item, so no day-count applies) — importantly distinct from a true 0 days, which means "applicable but zero days occurred".

Treatment: Recode 8 to NA for each field in a new _clean column, so the two meanings ('not applicable' vs 'zero days') are never conflated, which would otherwise bias any food-insecurity severity score built from these fields.

```
d07 <- d07 %>% mutate(
  FewFoodsDays_clean = if_else(FewFoodsDays == 8, NA_real_, FewFoodsDays),
  SkippedMealDays_clean = if_else(SkippedMealDays == 8, NA_real_, SkippedMealDays),
  AteLessDays_clean = if_else(AteLessDays == 8, NA_real_, AteLessDays),
  RanOutDays_clean = if_else(RanOutDays == 8, NA_real_, RanOutDays)
)
```

Effect: FewFoodsDays_clean gains 15,823 NAs; SkippedMealDays_clean gains 17,384; AteLessDays_clean gains 16,798; RanOutDays_clean gains 17,108.

Verification: sapply(..., function(x) sum(is.na(x))) on the four _clean columns returns exactly these four counts, confirming the sentinel-to-NA conversion for each field individually.

3.5.3 HouseholdSize == 0

Problem: 10 households are recorded with zero members in this file — a separate, independently-captured copy of HouseholdSize from D04's.

Treatment: Same treatment as D04 (§3.3.4): recode to NA in a new HouseholdSize_clean column; the rest of the row is retained.

```
d07 <- d07 %>% mutate(HouseholdSize_clean = if_else(HouseholdSize == 0, NA_real_,
HouseholdSize))
```

Effect: 10 values move from 0 to NA in D07's HouseholdSize_clean.

Verification: sum(HouseholdSize == 0, na.rm = TRUE) = 10.

3.5.4 Income Sentinel Family and Extreme Outliers

Problem: The value 9999999 (seven 9s) is used as a "refused/missing" sentinel in TotalMonthlyHouseholdIncomeRaw (2.1% of rows). Investigating the remaining distribution — rather

than assuming it was now clean — surfaced a wider family of literal repeated-nine placeholders at other lengths (99999999, 999999999, 9999999999: 8, 9 and 10 nines), the same convention used at different lengths rather than genuine incomes of R100 million–R10 billion. A rule matching only the exact 7-digit value would have missed these.

Treatment: A general helper function (is_repdigit_sentinel) recodes any value that is a run of 6 or more repeated 9s to NA in a new Income_clean column, catching the whole sentinel family at once. Values below that pattern are left untouched and treated as plausible, since a national household survey can genuinely include a few high-income households — the distinguishing feature used here is the digit pattern, not simply the magnitude.

```
is_repdigit_sentinel <- function(x, digit = "9", min_len = 6) {
  s <- as.character(as.integer(x))
  !is.na(x) & nchar(s) >= min_len & sapply(strsplit(s, ""), function(ch) all(ch == digit))
}
d07 <- d07 %>% mutate(Income_clean =
if_else(is_repdigit_sentinel(TotalMonthlyHouseholdIncomeRaw), NA_real_,
TotalMonthlyHouseholdIncomeRaw))
```

Effect: Income_clean summary: Min 0, 1st Qu. 2,470, Median 5,030, Mean 11,754, 3rd Qu. 11,418, Max 1,460,000, with 451 NAs. Of the remaining non-sentinel values, 9 households still report income above R500,000/month; these were inspected individually and are genuinely-shaped numbers (not repeating digits), so they are flagged for awareness rather than deleted.

Verification: summary(Income_clean) confirms the NA count and range above; d07 %>% filter(Income_clean > 500000) lists exactly 9 households (e.g. HouseholdID 599112930000010001 at R1,460,000) for downstream review.

3.5.5 RanOutOfFoodCode Recoding

Problem: Raw codes are 1 / 2 / 9, which are not self-explanatory for analysis or reporting.

Treatment: Recode into an analytically meaningful RanOutOfFood_label factor while preserving the original source code, following the typical Stats SA convention (1 = Yes, 2 = No), with 9 kept as its own explicit "Don't know/refused" category rather than merged into either Yes or No. This label mapping is flagged as provisional, pending confirmation against the official GHS codebook, which was not supplied with the data pack.

```
d07 <- d07 %>% mutate(RanOutOfFood_label = case_when(
  RanOutOfFoodCode == 1 ~ "Yes",
  RanOutOfFoodCode == 2 ~ "No",
  RanOutOfFoodCode == 9 ~ "Don't know/refused",
  TRUE ~ NA_character_
))
```

Effect: A new character column RanOutOfFood_label is added alongside the original numeric code.

Verification: table(RanOutOfFood_label, useNA = "ifany") returns: Don't know/refused = 29, No = 17,079, Yes = 3,832 — accounting for all 20,940 rows with no unexpected NAs introduced.

3.6 D09 — Household Resources

3.6.1 Exact Duplicate Rows

Problem: 15 exact duplicate rows found.

Treatment: Remove with distinct().

```
d09 <- d09 %>% distinct()
```

Effect: Row count falls from 20,955 to 20,940.

Verification: "D09 rows before: 20955 | after de-duplication: 20940 | removed: 15".

3.6.2 MainIncomeSourceCode

Problem: 65 genuine NAs, plus a separate 99 sentinel (4 rows) sitting outside the documented 1–8 category range.

Treatment: Genuine NAs are left as-is (no fabricated imputation). The 99 sentinel is recoded to NA separately in a new MainIncomeSourceCode_clean column.

```
d09 <- d09 %>% mutate(MainIncomeSourceCode_clean = if_else(MainIncomeSourceCode == 99,
NA_real_, MainIncomeSourceCode))
```

Effect: MainIncomeSourceCode_clean has the 4 sentinel rows converted to NA, in addition to the 65 pre-existing genuine NAs.

Verification: sum(is.na(MainIncomeSourceCode)) = 65; sum(MainIncomeSourceCode == 99, na.rm = TRUE) = 4 — both confirmed directly.

3.6.3 TotalVehicles Mixed Encoding

Problem: This column was read as character because it mixes genuine vehicle counts (1–7) with the sentinel 88, the sentinel 99, and the literal text "Unknown" — three different encodings of "not a real count" in one field, which also blocks any numeric summary until resolved.

Treatment: 88 is verified, via a cross-tab against VehicleAssetCode, to mean "no vehicle" rather than missing — households with VehicleAssetCode == 2 (no vehicle) show TotalVehicles == 88 in the overwhelming majority of cases (15,217 of 15,232) — so 88 is recoded to 0. 99 and "Unknown" have no such basis and are recoded to NA. The result, TotalVehicles_clean, is a genuine numeric column.

```
tv_num <- suppressWarnings(as.numeric(d09$TotalVehicles))
tv_num[tv_num == 88] <- 0
tv_num[tv_num == 99] <- NA
d09$TotalVehicles_clean <- tv_num
```

Effect: TotalVehicles_clean summary: Min 0, 1st Qu. 0, Median 0, Mean 0.41, 3rd Qu. 1, Max 16, with 30 NAs (10 "Unknown" + 8 remaining 99s + any genuinely non-numeric residual after coercion).

Verification: table(VehicleAssetCode, TotalVehicles == "88") confirms the 88↔"no vehicle" correspondence (15,217 of 15,232 no-vehicle households show 88); sum(TotalVehicles == "Unknown", na.rm = TRUE) = 10; sum(tv_num == 99, na.rm = TRUE) = 8; sum(tv_num == 88, na.rm = TRUE) = 15,220 before recoding.

3.6.4 TotalMonthlyHouseholdIncomeRaw

Problem: This file carries its own copy of the same income field used in D07 (§3.5.4), affected by the same repeated-9 sentinel family.

Treatment: Apply the identical is_repdigit_sentinel rule used for D07 so the two files are cleaned consistently rather than diverging into two different income-cleaning conventions.

```
d09 <- d09 %>% mutate(Income_clean =
if_else(is_repdigit_sentinel(TotalMonthlyHouseholdIncomeRaw), NA_real_,
TotalMonthlyHouseholdIncomeRaw))
```

Effect: Income_clean summary for D09: Min 0, 1st Qu. 2,470, Median 5,030, Mean 11,768, 3rd Qu. 11,408, Max 1,460,000, with 433 NAs.

Verification: sum(sentinel_income9) = 433, a clean numeric count (unlike the D07 run — see the implementation note in §3.5.4), consistent with the size of the sentinel population expected for this file.

3.7 Cross-File Consistency Checks (Diagnostic Only)

D04, D07 and D09 each carry their own copy of HouseholdSize, and D07/D09 each carry their own copy of TotalMonthlyHouseholdIncomeRaw. These duplicated fields are compared purely to document where they disagree — this is a verification join, not an analytical merge, and no value is overwritten based on it, since there is no codebook basis in this data pack for deciding which file's figure is "correct". Any household flagged below is left for Task 4 (Person 3) to review as a judgement call, not resolved automatically.

Finding: 22 households have a HouseholdSize that disagrees across at least two of D04/D07/D09, and 18 households have a TotalMonthlyHouseholdIncomeRaw that disagrees between D07 and D09.

Verification: nrow(hh_size) = 22 and nrow(income_check) = 18. Both result sets are exported (flag_householdsize_mismatches.csv, flag_income_mismatches.csv) for Task 3/4 to review rather than silently resolved in Task 2.

3.8 Post-Cleaning Verification Summary

Dataset	Rows (final)	Unique HouseholdID	Province categories
D01	70,415	20,940	9
D04	20,940	20,940	9
D05	20,925	20,925	9
D07	20,940	20,940	9
D09	20,940	20,940	9

D04, D07 and D09 show identical unique-HouseholdID counts (20,940), confirming they describe the same household population. D05's 20,925 unique households is a subset of that count, consistent with the 15 orphan records removed in §3.4.1. D01's unique-HouseholdID count (20,940) is lower than its row count (70,415) because it is person-level, and it already excludes the 25 rows with no HouseholdID at all (§3.2.1).

4. Task 3 — Integrate and Construct

4.1 Inputs

This stage builds directly on Person 1's Task 1/2 outputs and does not re-clean any data. The five cleaned exports (D01_cleaned.csv, D04_cleaned.csv, D05_cleaned.csv, D07_cleaned.csv, D09_cleaned.csv) and the two audit/flag files (flag_householdsize_mismatches.csv, flag_income_mismatches.csv) are read in directly, with HouseholdID again forced to character type.

Verification: Row counts on import match Person 1's final Task 2 counts exactly — D01: 70,415 | D04: 20,940 | D05: 20,925 | D07: 20,940 | D09: 20,940 — and the flag files carry through the same 22 HouseholdSize mismatches and 18 income mismatches reported in §3.7, confirming a clean, lossless handoff between Task 2 and Task 3.

4.2 Target Grain and Integration Strategy

The analytical grain is one row per HouseholdID. D04, D05, D07 and D09 are already at this grain (confirmed in §3.8). D01 is person-level and cannot legitimately be row-joined onto a household-level table — joining on HouseholdID alone would silently fan out the household rows (a one-to-many join), which is not a valid integration at this grain. The justified alternative is aggregation: D01 is collapsed to one summary row per household first, and only the aggregated table is joined.

4.3 Aggregate D01 to Household Grain

Problem: D01 is at person-level grain (70,415 rows across 20,940 households) and cannot be joined directly to the household-level tables without fanning out rows.

Treatment: Group D01 by HouseholdID and summarise into one row per household: HouseholdSize_D01 (person count), Age_mean and Age_median (from Age_clean), N_Dependants (members aged <15 or >64), Dependency_ratio (N_Dependants divided by the working-age remainder), and N_PopulationGroups (distinct PopulationGroupCode values in the household).

```
d01_agg <- d01 %>%
  group_by(HouseholdID) %>%
  summarise(
    HouseholdSize_D01 = n(),
    Age_mean = mean(Age_clean, na.rm = TRUE),
    Age_median = median(Age_clean, na.rm = TRUE),
    N_Dependants = sum(Age_clean < 15 | Age_clean > 64, na.rm = TRUE),
    Dependency_ratio = N_Dependants / pmax(HouseholdSize_D01 - N_Dependants, 1),
    N_PopulationGroups = n_distinct(PopulationGroupCode, na.rm = TRUE),
    .groups = "drop"
  )
```

Effect: d01_agg has exactly 20,940 rows — one per household — down from 70,415 person-level rows.

Verification: before_person_rows = 70,415; before_households (distinct HouseholdID) = 20,940; nrow(d01_agg) = 20,940. A stopifnot(nrow(d01_agg) == before_households) assertion passes, confirming the aggregation is exactly 1:1 at household grain and safe to join.

4.4 Confirm Join Cardinality Before Joining

Every household-level file is expected to be 1:1 on HouseholdID. This is checked explicitly for all five inputs before any join is attempted, rather than assumed.

File	Rows	Distinct HouseholdID	1:1?
D01 (aggregated)	20,940	20,940	TRUE
D04	20,940	20,940	TRUE
D05	20,925	20,925	TRUE
D07	20,940	20,940	TRUE
D09	20,940	20,940	TRUE

D05 is expected and confirmed to be a subset of the household population covered by D04/D07/D09 — 15 D04 households have no matching D05 record (checked explicitly via an anti-join rather than assumed), consistent with the 15 orphan D05 rows removed in §3.4.1. Left, rather than inner, joins are therefore used so those households are retained with NA D05 fields rather than silently dropped.

4.5 Sequential Joins with Row-Count Verification

Joins are performed one at a time — not chained in a single pipe — so the row count can be checked after each step. D04 is used as the base table, since it is confirmed 1:1 and dwelling-level. D07 and D09 each carry their own copies of HouseholdSize, HouseholdSize_clean, Income_clean and TotalMonthlyHouseholdIncomeRaw; because Task 2 flagged mismatches between these copies with no codebook basis to prefer one file's figure over another, every colliding column is renamed explicitly before joining (e.g. HouseholdSize_D07, Income_clean_D07) rather than letting dplyr's automatic suffixing decide silently based on join order.

Join step	Rows before	Rows after	Row count preserved?
1. D04 + D01_agg (left join)	20,940	20,940	TRUE
2. + D05 (left join)	20,940	20,940	TRUE
3. + D07 (left join, renamed collisions)	20,940	20,940	TRUE
4. + D09 (left join, renamed collisions)	20,940	20,940	TRUE

Verification: The final household_data table has 20,940 rows and 20,940 distinct HouseholdIDs (stopifnot(nrow(household_data) == n_distinct(household_data$HouseholdID)) passes). The row count held constant at every step and matched the distinct-household count throughout, confirming every join was 1:1 with no fan-out, and that no household was duplicated or unintentionally dropped.

4.6 Column-Level Selection: Dropping Redundant Raw Income Copies

Problem: IncomeRaw_D07 and IncomeRaw_D09 are the pre-sentinel-cleaned income figures — the same fields Income_clean_D07 and Income_clean_D09 were derived from in Task 2. Keeping both the raw and cleaned versions carries two redundant, sentinel-contaminated columns into the final dataset for no analytical benefit.

Treatment: Drop IncomeRaw_D07 and IncomeRaw_D09, mirroring Person 1's §2.4 column-level selection approach of dropping columns with no analytical content once a cleaned equivalent exists.

```
household_data <- household_data %>% select(-IncomeRaw_D07, -IncomeRaw_D09)
```

Effect: household_data shrinks from 132 to 130 columns.

Verification: "Columns before: 132 | after dropping raw income duplicates: 130".

4.7 Derived / Engineered Features

Three new analytical features are engineered onto household_data, each with its source columns, construction logic and rationale stated explicitly.

4.7.1 Services_access_index

Problem: There is no single existing field capturing overall basic-service access; water and sanitation quality are each split across several coded fields.

Treatment: Sum two binary indicators — piped water present (PipedWaterCode_clean == 1) and adequate sanitation (MainToiletCode in {1, 2}) — into a 0–2 index. Electricity access is not captured in D04 or D05 in this data pack (confirmed against both files' column lists), so it is deliberately excluded rather than substituted with a proxy such as D09's appliance-ownership codes, which measure something conceptually different — a documented limitation rather than a fabricated component.

```
Services_access_index = rowSums(cbind(
  PipedWaterCode_clean == 1,
  MainToiletCode %in% c(1, 2)
), na.rm = TRUE)
```

Effect: A new integer column Services_access_index (0–2) is added.

Verification: summary(Services_access_index) returns Min 0, 1st Qu. 0, Median 1, Mean 0.69, 3rd Qu. 1, Max 2 — a plausible distribution with no out-of-range values.

4.7.2 Income_per_capita

Problem: Household income alone does not account for household size, and D07 and D09's income figures were flagged in Task 2 as mismatched for some households with no codebook basis to prefer one over the other.

Treatment: Divide D07's cleaned income (Income_clean_D07) by the D01-derived household size (HouseholdSize_D01), using pmax(., 1) to avoid division by zero. D07's income figure is used rather than D09's for consistency with the food-security variables this feature is meant to relate to.

```
Income_per_capita = Income_clean_D07 / pmax(HouseholdSize_D01, 1)
```

Effect: A new double column Income_per_capita is added.

Verification: summary(Income_per_capita) returns Min 0, 1st Qu. 880, Median 1,952, Mean 4,572, 3rd Qu. 4,474, Max 365,000, with 451 NAs — the NA count matches Income_clean_D07's own NA count from Task 2 (§3.5.4), confirming no additional missingness was introduced by the division.

4.7.3 FoodInsecure_flag

Problem: RanOutOfFood_label (§3.5.5) is a three-level factor, which is less convenient than a binary flag for many modelling/segmentation approaches.

Treatment: Derive a clean 0/1 convenience flag from the recoded label (1 when RanOutOfFood_label == "Yes", else 0), while retaining the original three-level factor for analyses that need the "Don't know/refused" distinction.

```
FoodInsecure_flag = if_else(RanOutOfFood_label == "Yes", 1L, 0L)
```

Effect: A new integer column FoodInsecure_flag is added.

Verification: table(FoodInsecure_flag, useNA = "ifany") returns 0 = 17,108 and 1 = 3,832 — the 3,832 count matches the "Yes" count from §3.5.5 exactly, confirming the flag was derived correctly.

4.8 Final Data Dictionary

Retained and engineered variables are tiered qualitatively following Person 1's §2.3 scheme (High = directly measures resources/access hypothesised to drive the outcome; Medium = demographic/context variable that conditions the outcome; Low = administrative/design field, not a substantive predictor).

Variable	Source	Type	Tier	Description
HouseholdID	D04/D05/D07/D09	character	Key	Household key, join grain
HouseholdSize_D01	D01 (aggregated)	integer	Medium	Count of persons per household from D01
Age_mean	D01 (aggregated)	double	Medium	Mean cleaned age of household members
Age_median	D01 (aggregated)	double	Medium	Median cleaned age of household members
N_Dependants	D01 (aggregated)	integer	Medium	Members aged <15 or >64
Dependency_ratio	D01 (engineered)	double	Medium	N_Dependants / working-age members
N_PopulationGroups	D01 (aggregated)	integer	Medium	Distinct PopulationGroupCode values within household
HeadPopulationGroupCode	D04	integer	Medium	Population group of household head
HeadSexCode	D04	integer	Medium	Sex of household head
HeadAge	D04	integer	Medium	Age of household head
MainDwellingCode	D04	integer	Medium	Main dwelling type
WallMaterialCode_clean	D04	integer	Medium	Cleaned wall material code
RoofMaterialCode	D04	integer	Medium	Roof material code
FloorMaterialCode	D04	integer	Medium	Floor material code
WallConditionCode	D04	integer	Medium	Wall condition code
RoofConditionCode	D04	integer	Medium	Roof condition code
FloorConditionCode	D04	integer	Medium	Floor condition code
TotalRooms_clean	D04	integer	Medium	Cleaned total rooms in dwelling
TenureCode	D04	integer	Medium	Tenure status code
RentRaw_clean	D04	double	Medium	Cleaned monthly rent
RDPHousingCode	D04	integer	Medium	RDP housing flag
HousingSubsidyCode	D04	integer	Medium	Housing subsidy flag

Variable	Source	Type	Tier	Description
HouseholdSize_clean	D04	integer	Medium	D04's cleaned household size (base-table copy)
MetroCode	D04	integer	Low	Metro/non-metro geography code
ProvinceClean	D04	character	Low	Canonical province label (base-table copy, §3.1)
PipedWaterCode_clean	D05	integer	High	Cleaned piped-water access code
WaterSafeCode	D05	integer	High	Perceived water safety code
WaterTreatmentCode	D05	integer	Medium	Water treatment method code
MainToiletCode	D05	integer	High	Main toilet type code
SharedToiletCode	D05	integer	Medium	Shared-toilet flag
ToiletDistanceCode_clean	D05	integer	Medium	Cleaned distance-to-toilet code
HandwashingFacilityCode	D05	integer	Medium	Handwashing facility availability code
WaterInterruptionFrequencyCode_clean	D05	integer	Medium	Cleaned water-interruption frequency code
AdultHungerCode	D07	integer	High	Adult hunger indicator code
ChildHungerCode	D07	integer	High	Child hunger indicator code
WorriedFoodWouldRunOutCode	D07	integer	High	Food-insecurity worry indicator code
UnableToEatHealthyFoodCode	D07	integer	High	Diet-quality food-insecurity code
FewFoodsDays_clean	D07	integer	High	Cleaned days eating few food types
SkippedMealDays_clean	D07	integer	High	Cleaned days meals were skipped
AteLessDays_clean	D07	integer	High	Cleaned days ate less than needed
RanOutDays_clean	D07	integer	High	Cleaned days food ran out
RanOutOfFoodCode	D07	integer	High	Raw ran-out-of-food code, Stats SA convention 1/2/9
RanOutOfFood_label	D07	character	High	Recoded label (Yes/No/Don't know-refused); provisional pending GHS codebook
HouseholdSize_D07	D07	integer	Low	D07's own HouseholdSize copy, kept for transparency
HouseholdSize_clean_D07	D07	integer	Low	D07's own cleaned HouseholdSize copy

Variable	Source	Type	Tier	Description
MainIncomeSourceCode_clean	D09	integer	High	Cleaned main income source code
SocialGrantRecipients	D09	integer	High	Number of social grant recipients in household
TotalGrantRecipients	D09	integer	High	Total grant recipients in household
SalaryEarnersInHousehold	D09	integer	High	Number of salary earners in household
VehicleAssetCode	D09	integer	Medium	Vehicle ownership indicator code
TotalVehicles_clean	D09	integer	Medium	Cleaned total vehicle count
FridgeAssetCode	D09	integer	Medium	Fridge ownership indicator code
FreezerAssetCode	D09	integer	Medium	Freezer ownership indicator code
HouseholdSize_D09	D09	integer	Low	D09's own HouseholdSize copy, kept for transparency
Income_clean_D07	D07	double	High	D07's cleaned income; authoritative income figure (§4.7.2)
Income_clean_D09	D09	double	Low	D09's cleaned income, kept for transparency, not used in Income_per_capita
Services_access_index	D05 (engineered)	integer	High	Count of adequate water+sanitation indicators (0-2); electricity excluded
Income_per_capita	D07 (engineered)	double	High	Income_clean_D07 / HouseholdSize_D01
FoodInsecure_flag	D07 (engineered)	integer	High	1 = RanOutOfFood_label == 'Yes', else 0

5. Task 4 — Prepare Data for Modelling

5.1 Define the Modelling Target

Problem: Milestone 3's predictive goal needs a single binary modelling target rather than a raw multi-level food-security code.

Treatment: Adopt FoodInsecure_flag as the modelling target — it was already engineered in Task 3 (§4.7.3) from RanOutOfFoodCode, via the RanOutOfFood_label recode created in Task 2 (§3.5.5), rather than deriving a new target field from scratch.

Effect: No new column is created; FoodInsecure_flag (0 = food secure, 1 = food insecure) is carried forward unchanged from household_data as the target variable.

Verification: Presence and class counts are confirmed in household_data from Task 3 — see the §5.2 imbalance check below (17,108 vs 3,832), which are the same counts reported in §4.7.3.

5.2 Assess Class Imbalance

Problem: Is the target skewed enough to need resampling before modelling?

Treatment: Check the ratio between the majority class (food-secure) and minority class (food-insecure) before deciding on any rebalancing strategy.

Effect: Majority : minority ratio ≈ 4.46 : 1 (consistent with the 17,108 vs 3,832 split reported for FoodInsecure_flag in §4.7.3).

Verification: A ratio of ~4.46:1 is judged moderate rather than severe (contrasted explicitly with a severe case such as 99:1), so it is addressed through a stratified train/test split (§5.7) rather than a resampling technique such as SMOTE.

5.3 Remove Identifiers and Low-Value Columns

Problem: Redundant or low-value columns — per the team data dictionary's Tier column (§4.8) — would be meaningless, or even misleading, as model predictors.

Treatment: Drop every variable tagged "Key" or "Low" tier in the data dictionary.

5.4 Remove Leakage Variables

Problem: FoodInsecure_flag is built directly from RanOutOfFoodCode. Any variable drawn from the same food-security question battery is too directly tied to the target and would leak the answer to the model.

Treatment: Remove all 18 variables related to food security.

5.5 Transformations and Encoding

Problem: Category codes are stored as integers, which would make R treat them as continuous numbers rather than as categories.

Treatment: Convert every *Code column to a factor.

5.6 Scaling

Problem: K-means uses distance calculations, so numeric predictors that vary widely in scale (e.g. income in the thousands vs a 0–5 index) would let income dominate the distance metric purely due to scale, not real importance.

Treatment: Standardise (z-score) all numeric predictors.

5.7 Train/Test Split

Problem: Milestone 3 models need unseen data held out for evaluation, without the split accidentally distorting the target's class balance.

Treatment: Stratified 80/20 split, with seed = 381 set for reproducibility.

5.8 Leakage Prevention Check

Problem: Confirm whether removing the leakage and tier-based variables (§5.3–§5.4) changed the dataset's overall missingness, and document what remains for the group to be aware of.

6. Summary and Outstanding Items

Across Task 1–4, five source datasets were reconfirmed, cleaned of 61 duplicate rows, 25 unlinkable person records and dozens of sentinel/placeholder codes, integrated into a single 20,940-row, 130-column household-level analytical table with three engineered features and a full data dictionary, and then carried into a modelling-preparation pass that defines the target, checks class balance, removes leakage, and sets up encoding, scaling and a train/test split. The following items are carried forward as open items rather than resolved silently:

- 22 households with disagreeing HouseholdSize across D04/D07/D09, and 18 with disagreeing income between D07/D09 (flag_householdsize_mismatches.csv, flag_income_mismatches.csv) — no codebook basis exists in this data pack to prefer one file's figure over another.
- 9 households in D07 with cleaned income above R500,000/month, retained as plausible but flagged for review.
- The RanOutOfFood_label mapping (1 = Yes, 2 = No, 9 = Don't know/refused) is provisional pending confirmation against the official GHS codebook, which was not supplied with the data pack.
- Services_access_index deliberately excludes electricity access, since it is not captured in D04 or D05 in this data pack.